

(POSTER) Multi-Agent Framework for Decentralized Digital Twins

Mengbing Zhou¹, Zhiming Zhao^{1,2*}

¹ Institute of Informatics, University of Amsterdam, Amsterdam, Netherlands

² LifeWatch ERIC, Virtual Lab Innovation Center, Amsterdam, Netherlands

{m.zhou2, z.zhao}@uva.nl

* Corresponding author

Abstract—Digital Twin systems are increasingly evolving toward large-scale, distributed deployments, where scalable and time-sensitive processing becomes challenging under traditional centralized designs. This work presents a multi-agent framework for decentralized DTs, in which each DT service is paired with a configurable agent that manages task scheduling, coordination, and optional offloading to a Function-as-a-Service platform via a publish/subscribe messaging layer. By decoupling communication, computation, and control, the proposed architecture enables flexible task placement, scalable coordination, and adaptive execution across heterogeneous resources. A prototype implementation and preliminary experiments are presented to illustrate the feasibility of the approach.

Index Terms—Digital Twin, Multi-Agent Systems, Decentralized Architecture, Function-as-a-Service, Task Offloading

I. INTRODUCTION

Digital Twin (DT) technology is increasingly adopted to enable real-time monitoring, analysis, and control of complex cyber-physical and IoT systems. As the number of devices and the granularity of modeled entities grow, deployments are evolving from a few centralized instances to large-scale DT ecosystems. In such settings, DT services must process heterogeneous data streams and perform computation-intensive analytics under strict timing constraints, making scalability under temporal guarantees a key challenge [1].

In this work, we focus on *decentralized* DT systems. Decentralized DT systems distribute DT instances across edge and cloud nodes, where each instance maintains local state and decision logic and coordinates with others via message-based interactions without a global orchestrator [2]. Each DT interacts with other DTs and services as a peer to exchange data, delegate tasks, and achieve system-level goals [3]. This decentralization is motivated by scalability, resilience, data locality, and latency-sensitive multi-stakeholder requirements, but existing platforms provide limited support for managing decentralized coordination and timing constraints in a systematic way.

A *multi-agent* design is a promising direction for addressing these challenges. In a multi-agent system (MAS), an *agent* is an autonomous software entity that maintains local state, perceives its environment, communicates with other agents, and makes decisions to pursue its goals. Decentralized DT deployments raise MAS-style challenges: (i) coordinating many DTs without a central orchestrator,

(ii) making local scheduling and offloading decisions under heterogeneous, time-sensitive workloads with strict latency constraints, and (iii) adapting to dynamic changes in load, topology, and resource availability. These challenges naturally align with MAS capabilities, including decentralized coordination via explicit communication, local decision-making based on partial system views, and policy-driven adaptation to dynamic environments [4].

We therefore explore a *multi-agent framework* for decentralized DTs as part of the Dutch Research Council (NWO) Large-Scale Research Infrastructures (LSRI) programme for the LTER-LIFE infrastructure. In our design, each DT service is associated with a dedicated, homogeneous, and configurable DT agent; adding a new DT service requires only configuring its agent. DT agents mediate interactions with devices and other DTs via a publish/subscribe messaging middleware, maintain local world state, and decide whether to execute tasks locally or offload them to a Function-as-a-Service (FaaS) backend. On the FaaS side, a corresponding FaaS agent manages offloaded tasks and returns results to the originating DT agents. By routing interactions through agents rather than direct service-to-service coupling, the framework supports decentralized operation while providing a natural locus for embedding scheduling, offloading, and adaptation policies.

This work presents the design of the multi-agent DT framework, including the interaction between DT agents, the FaaS agent, and the messaging middleware, together with a prototype implementation and initial experiments. We discuss how this architecture can improve scalability and timing behavior in decentralized DT deployments and outline ongoing work on advanced scheduling and larger-scale evaluations.

II. SYSTEM OVERVIEW

Fig. 1 illustrates the proposed multi-agent framework for decentralized DTs, organized into three layers: (i) physical devices, (ii) a publish/subscribe messaging middleware (Kafka in the current implementation), and (iii) a compute layer hosting DT services, DT agents, and a FaaS platform.

At the device layer, heterogeneous devices (e.g., sensors, cameras, controllers) communicate with the system via an IoT gateway, which translates device protocols to and from

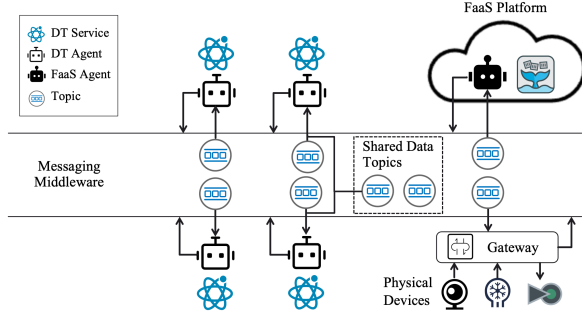


Fig. 1. System overview of the proposed multi-agent DT framework.

Kafka messages. Device data are published to Kafka topics (e.g., per-DT inbox or shared data topics), while control commands are routed back to devices via gateway topics. This design decouples device connectivity from DT logic and enables scalable integration of heterogeneous devices.

At the compute layer, each DT service is paired with a configurable DT agent, forming a DT instance. Each instance is associated with a private inbox topic that aggregates incoming messages. The DT agent consumes messages from its inbox and, when configured, from shared data topics, maintains the world state, and determines task execution according to its configuration. Agents are homogeneous and lightweight: adding a new DT service requires only instantiating a corresponding agent and configuring its actions and subscriptions. Agents may also exchange information via shared data topics to disseminate reusable state across DT instances. All outgoing messages from DT services are routed through their agents to appropriate Kafka topics (e.g., other DT inboxes, shared data, or gateway topics), enabling decentralized coordination without direct service coupling.

For elastic execution, DT agents may offload selected tasks to a FaaS platform. A FaaS agent mediates this process by consuming offload requests, scheduling function execution, and publishing results back to Kafka for retrieval by DT agents. By centralizing communication through Kafka and encapsulating coordination, scheduling, and offloading within agents, the framework supports flexible task placement, scalable coordination, and decentralized operation while aiming to preserve timing guarantees.

III. DT AGENT ARCHITECTURE

Fig. 2 shows the architecture of a DT agent and its interaction with the co-located DT service. Each DT service is paired with a homogeneous, configurable agent that serves as the single ingress and coordination point. The agent consumes messages from its private inbox topic and, optionally, from shared data topics. Incoming messages are processed by the *Ingress Handler*, which performs parsing and validation, and classifies them into task messages or context updates. Task messages are enqueued into *Msg Queues*, while context updates are directly applied to the *World State*. The agent behavior is governed by a *Configuration & Policy* module specifying actions, priorities, and

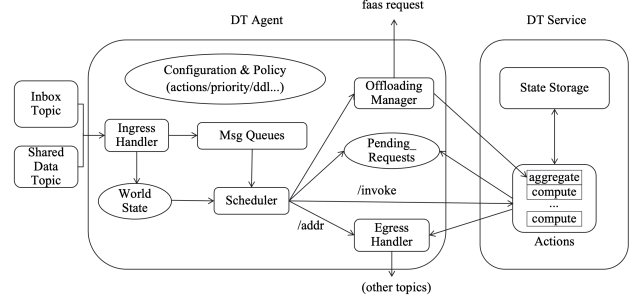


Fig. 2. Internal architecture of a DT agent and its co-located DT service.

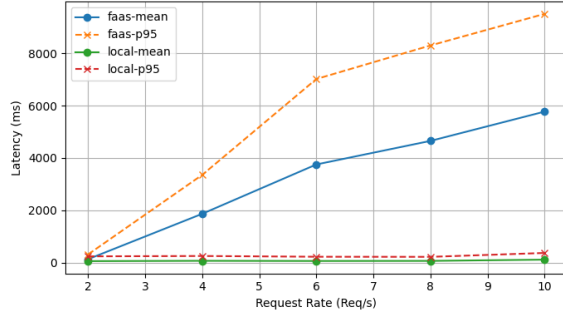
deadlines. The *World State* maintains context for scheduling decisions, including device observations, shared data, and load indicators. It is updated by the Ingress Handler and DT service actions, and queried by other components. Based on this state, the *Scheduler* periodically inspects the queues and determines task placement. Before execution, it may enrich task inputs with relevant state information.

Tasks selected for local execution are dispatched to the DT service's *Actions*, including (i) *compute* functions for local processing and (ii) *aggregate* functions that integrate intermediate and FaaS results into *State Storage*. Aggregated results may further update the World State or be routed by the *Egress Handler* to generate responses or trigger subsequent services. Tasks selected for remote execution are handled by the *Offloading Manager*, which packages requests, records metadata, and publishes them to Kafka. Returned FaaS results are processed and forwarded to aggregate actions for state integration. For critical requests requiring acknowledgment, the agent records them in a *Pending Requests* table. The corresponding FaaS function or local service later returns a confirmation message. Together, these mechanisms enable reliable coordination while keeping DT services decoupled from messaging details.

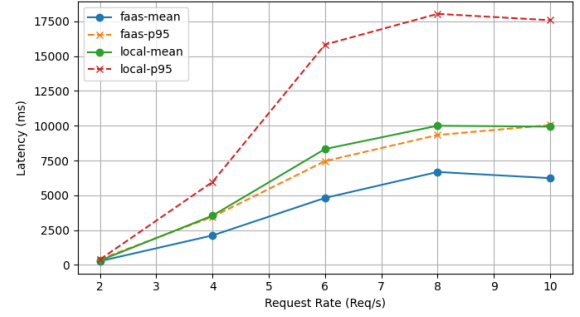
IV. PRELIMINARY EVALUATION

We have implemented a prototype of the proposed framework, including DT agents, the FaaS agent, a Kafka-based messaging layer, and two DT service simulations with their Actions. The prototype is deployed on four KVM-based virtual machines with Intel Xeon Cascade Lake processors running Ubuntu 24.04 LTS. One VM (2 vCPUs, 3.8 GB memory) hosts the Kafka broker, and another VM with the same configuration runs the lightweight FaaS runtime *faasd* together with the FaaS agent. The remaining two VMs (each with 1 vCPU and 1.9 GB memory) host one DT instance each, consisting of a DT service and its DT agent.

We evaluate the system using two image-processing DT services based on the ImageNet-1k dataset [5]. The first service performs image resizing, while the second performs image classification using a ResNet50 model [6]. Each service can execute locally or be offloaded as a FaaS function deployed on the *faasd* node. In the experiments, a client sends 100 requests to the resize service at fixed rates. Each



(a) Resize service.



(b) Classification service.

Fig. 3. Mean and 95th-percentile end-to-end latency under different request rates, comparing all-local and all-offloaded execution modes. For the classification service at 10 req/s, only 76 (all-local) and 44 (all-offloaded) requests complete; reported latencies are computed over completed requests.

completed resize triggers a classification request, resulting in 100 classification invocations. For each service, we compare two execution modes: (i) *all-local*, where tasks are executed within the DT service, and (ii) *all-offloaded*, where tasks are executed via the FaaS platform. We measure end-to-end latency from request generation to completion.

Fig. 3 shows the mean and 95th-percentile latencies under different request rates. Latency increases with load due to queuing effects in DT agents and the FaaS platform. For the lightweight resize service, local execution remains stable, while offloading degrades due to invocation overhead. For the compute-intensive classification service, offloading consistently outperforms local execution on resource-constrained DT hosts. At higher rates (e.g., 10 req/s), some classification requests are dropped, and reported latencies reflect only completed requests.

These results provide initial insights for system design. They indicate that a static execution strategy is insufficient for heterogeneous workloads and highlight the need for adaptive, policy-driven scheduling within DT agents. In particular, lightweight tasks should be executed locally to avoid offloading overhead, while compute-intensive tasks benefit from remote execution. This motivates the development of dynamic scheduling and offloading strategies that leverage the agent’s world state for context-aware decision making.

V. FUTURE DIRECTIONS

We plan to progressively evolve the proposed framework along three main directions. First, we will extend DT agent capabilities to support richer coordination patterns, more flexible use of shared data topics, and finer-grained world-state management. This will enable more complex inter-DT interactions beyond the current simple task chain. Second, we will design timing-aware scheduling and offloading strategies. Building on insights from our preliminary experiments, these strategies will exploit world-state information to make adaptive execution decisions and better satisfy latency constraints. Third, we will scale the system to larger deployments, including more DT agents, larger messaging

clusters, and multiple FaaS backends, and evaluate alternative pub/sub middleware systems with different performance and reliability characteristics. This will allow us to assess scalability, coordination efficiency, and timing guarantees under realistic and heterogeneous deployment settings.

In addition, we will investigate how existing multi-agent technologies can be integrated into the framework. Rather than implementing all agent functionality from scratch, we aim to leverage mature platforms for communication, coordination, and policy management, and adapt them to the requirements of decentralized DT systems, such as timing guarantees and data-driven decision making.

ACKNOWLEDGMENT

This research was partially supported by the Dutch Research Council (NWO) Large-Scale Research Infrastructures (LSRI) programme for the LTER-LIFE infrastructure (grant 184.036.014), LifeWatch ERIC, and the European Union through the projects ENVRI-Hub Next (101131141), EVERSE (101129744), BlueCloud-2026 (101094227), OSCARS (101129751), and BMD (101181294). ChatGPT was used to improve the readability of this manuscript.

REFERENCES

- [1] S. Mihai, M. Yaqoob, D. V. Hung, W. Davis, P. Towakel, M. Raza, M. Karamanoglu, B. Barn, D. Shetve, R. V. Prasad, H. Venkataraman, R. Trestian, and H. X. Nguyen, “Digital twins: A survey on enabling technologies, challenges, trends and future prospects,” *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2255–2291, 2022.
- [2] Y. Cao, J. Cao, B. Du, and R. Li, “Decentralized digital twin networks,” *IEEE Communications Magazine*, vol. 64, no. 2, pp. 90–96, 2026.
- [3] P. Glass and G. Di Marzo Serugendo, “Coordination model and digital twins for managing energy consumption and production in a smart grid,” *Energies*, vol. 16, no. 22, 2023.
- [4] Y. Kalyani and R. Collier, “The role of multi-agents in digital twin implementation: Short survey,” *ACM Comput. Surv.*, vol. 57, no. 3, Nov. 2024.
- [5] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “ImageNet Large Scale Visual Recognition Challenge,” *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.